🛡️ Module 11: Reverse Engineering, Fuzzing and Binary Exploitation

```
# Exercise 04: Performing Advanced Binary Analysis
```

Scenario

Advanced Binary Analysis and Code Obfuscation involves analyzing techniques designed to impede automated reverse engineering tools and static linear disassemblers. One such technique is **Instruction Overlapping**, which exploits variable-length instruction encodings in x86/x86-64 architectures. Because x86/x86-64 instructions are not byte-aligned, an attacker or software author can embed a conditional jump instruction (`jne` / `jnz`) that targets a byte offset located in the *middle* of an existing multi-byte instruction.

When a standard disassembler processes code sequentially (linear sweep or basic block disassembly), it decodes the primary instruction branch and skips the hidden bytes. However, when executed at runtime with a specific register state, the CPU jumps directly into the middle of the instruction byte sequence, decoding an entirely different, valid instruction sequence that alters execution flow (such as modifying return values or injecting hidden functionality).

In this exercise, we log into **Software-Test-Linux** (**student** / **studentpassword**), author an overlapping instruction C source file (**overlap.c**), compile it using GCC (**gcc -o overlap overlap.c**), inspect the binary format (**elf64-x86-64**), and disassemble specific memory offsets using **objdump** with Intel syntax (**objdump -M intel --start-address=0x4005f6 -d overlap** and **objdump -M intel --start-address=0x400612 -d overlap**) to reveal both the visible linear code stream and the obfuscated overlapping instruction branch.

Objective

The objective of this lab is to help students learn how to:
- Authenticate to target 64-bit Linux VM (**student** / **studentpassword**)
- Understand variable-length x86/x86-64 instruction decoding and opcode alignment properties
- Create an obfuscated C application (**overlap.c**) utilizing inline assembly and overlapping jump targets
- Compile binary executable using GCC: **gcc -o overlap overlap.c**
- Identify target binary ELF file format: **elf64-x86-64**
- Disassemble standard linear execution block starting at base address **0x4005f6** using **objdump -M intel --start-address=0x4005f6 -d overlap**
- Analyze conditional jump `jne` targeting mid-instruction byte offset **0x400612**
- Uncover obfuscated overlapping hidden instructions by disassembling custom entry point: **objdump -M intel --start-address=0x400612 -d overlap**

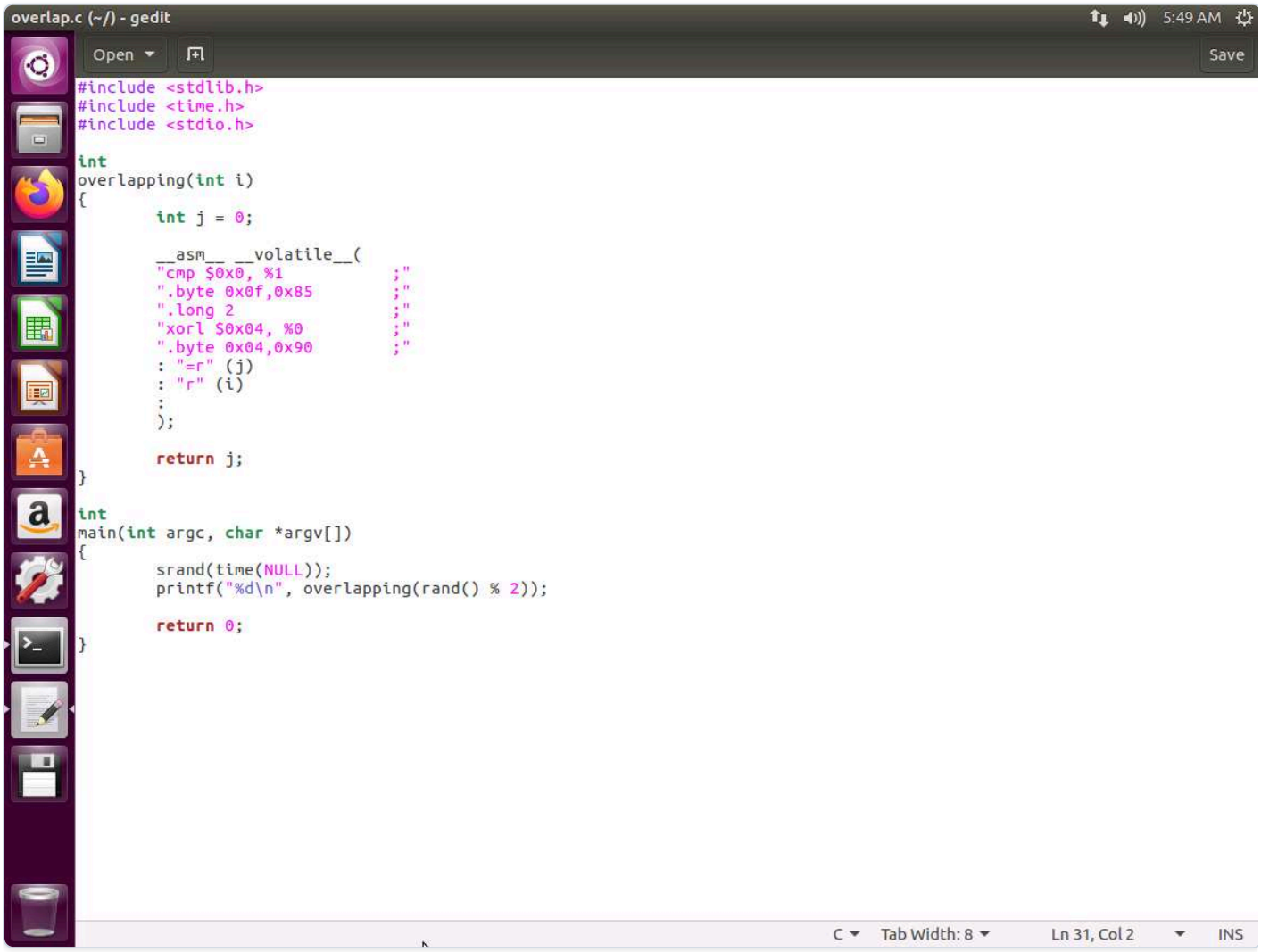
Task 1: Construct Obfuscated Overlapping Instruction Binary and Uncover Hidden Branch via Objdump

In this task, we log into the 64-bit Linux system, write `overlap.c`, compile it via `gcc`, inspect the file format (`elf64-x86-64`), perform targeted disassembly at offset `0x4005f6`, and disassemble the hidden overlapping branch at offset `0x400612` .

Lab Tasks

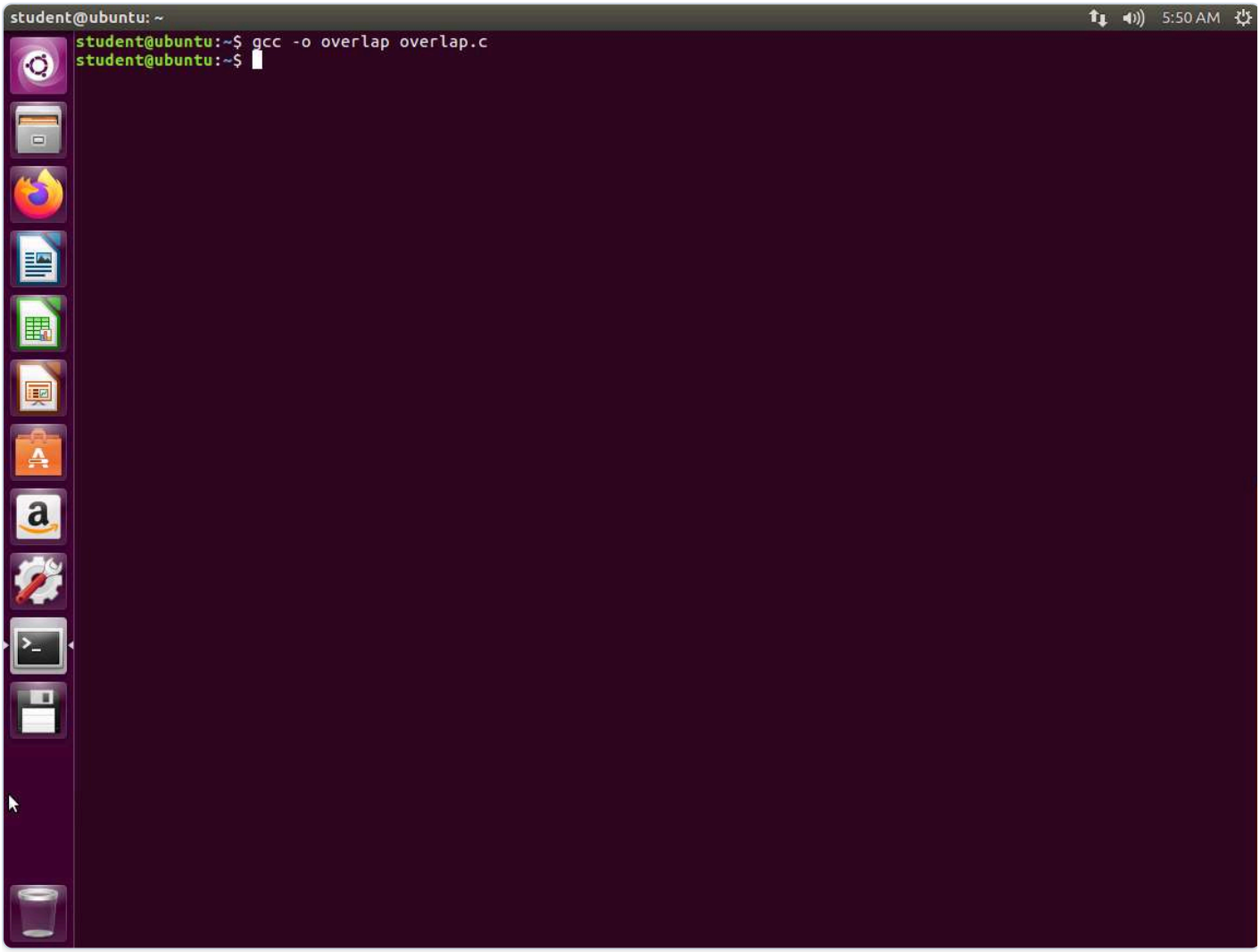
1. Log into **Software-Test-Linux-32bit** / **Software-Test-Linux** VM using credentials (**student** / **studentpassword**).

2. Create C source file **overlap.c** containing inline assembly instruction overlap logic:

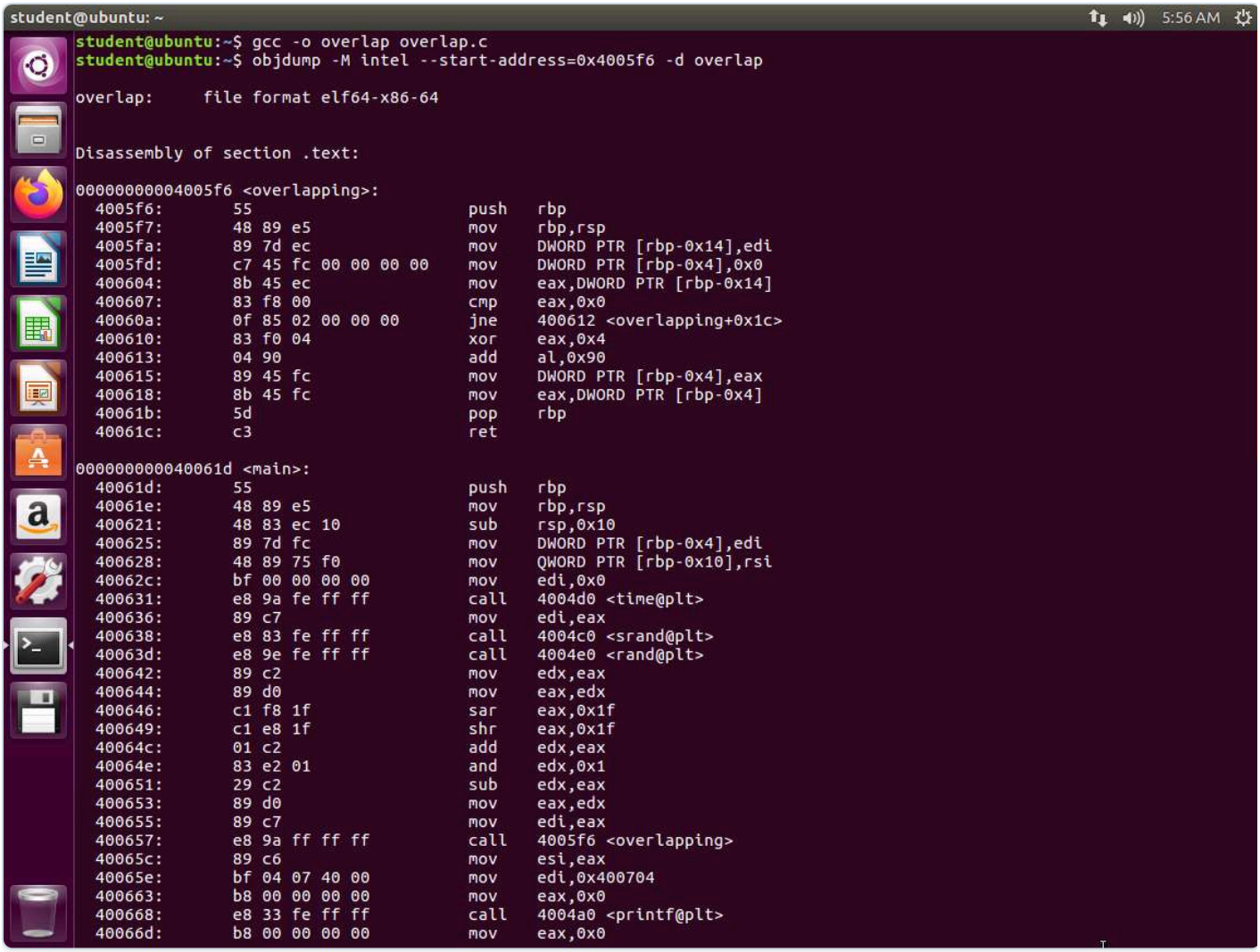


1. Compile the C source file into a 64-bit binary using GCC:

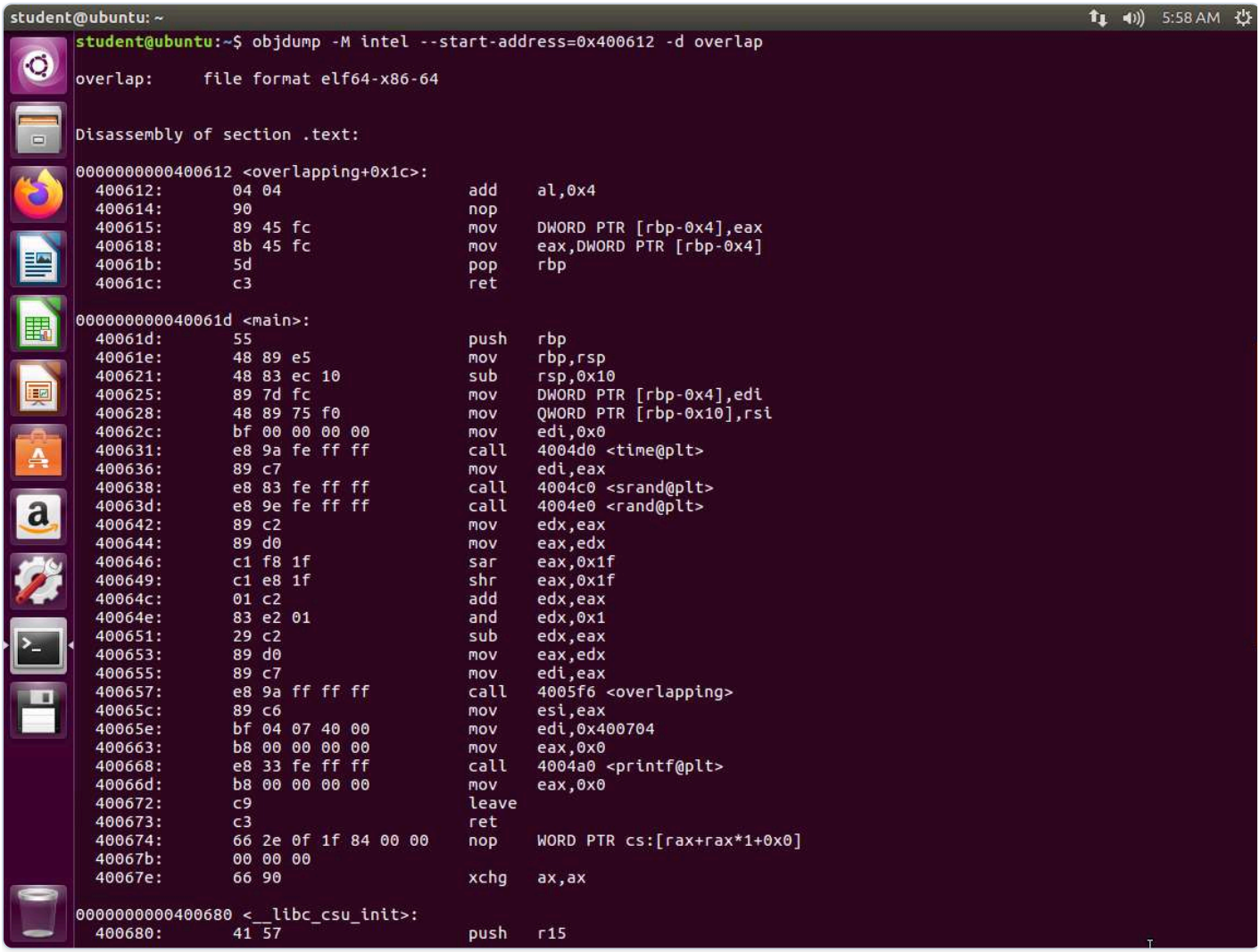
   gcc -o overlap overlap.c



1. Disassemble the primary linear code block starting at address **0x4005f6** using Intel syntax:
   objdump -M intel --start-address=0x4005f6 -d overlap
   Observe identified binary file format: **elf64-x86-64**.
   Note `jne` *instruction jumping into mid-instruction address offset* **0x400612**.



1. Disassemble from hidden offset **0x400612** to reveal obfuscated overlapping instruction branch:
   objdump -M intel --start-address=0x400612 -d overlap
   Observe hidden branch instructions (`add al, 0x4` *) executed when condition* `i != 0` *is satisfied.*



This concludes the lab exercise.

IMPORTANT: ❓ **Question 11.4.1.1** In Software-Test-Linux machine write and review a simple process of code obfuscation that deploys the method of instruction overlapping. Identify the file format that was identified during the review of the code in this lab. [!SUCCESS] ✅ **Answer:** **elf64-x86-64**